

▼ Principal Components Analysis(PCA)

PCA (Principal Components Analysis) is a dimensionally reduction technique used to reduce the no of features while retaining most of the information in the dataset.

Principal Components

Principal Components are new variables created from the original features.

PC1 (First Principal Component)

- Captures the maximum variance in the data.
- most important component

(Second Principal Component)

- Captures the remaining variance.
- Perpendicular to PC1.

PC3,PC4,...

- Captures progressively less variance.

Example

Original features

- Height
- Weight
- Age
- Income

After PCA

- PC1
- PC2

Explained variance Ratio

It helps how much information each principal component relations.

Example

Component	Variance Explained
PC1	70%
PC2	20%
PC3	7%
PC4	3%

Interpretation

- PC1 contains 70% of the information
- PC2 contains 20% of the information
- PC1 + PC2 contain 90% of the information

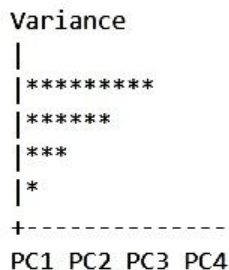
So we can keep only: PC1 + PC2

Screen plot

A screen plot shows:

Principal components vs Explained Variance

example



Purpose of Screen plot

- find how many components to keep
- Identify the "elbow point"
- Components after the elbow contribute very little information

Example

if:

When to apply

- Data visualization
- Reduce overfitting
- Highly correlated feature
- Faster training needed

Wen to apply

- Data visulazation
- Reduce overfitting
- Highly correlated feature
- Faster training needed

PCA Workflow

```
Data
|
Standarization
|
PCA
|
Principal components
|
Train model
```

```
[1]: from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import pandas as pd

# Load dataset
iris = load_iris()
X = iris.data

# Standardize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Load dataset
iris = load_iris()
X = iris.data

# Standardize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# Explained Variance Ratio
print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)

# PCA DataFrame
df = pd.DataFrame(

    X_pca,

    columns=["PC1", "PC2"]

)

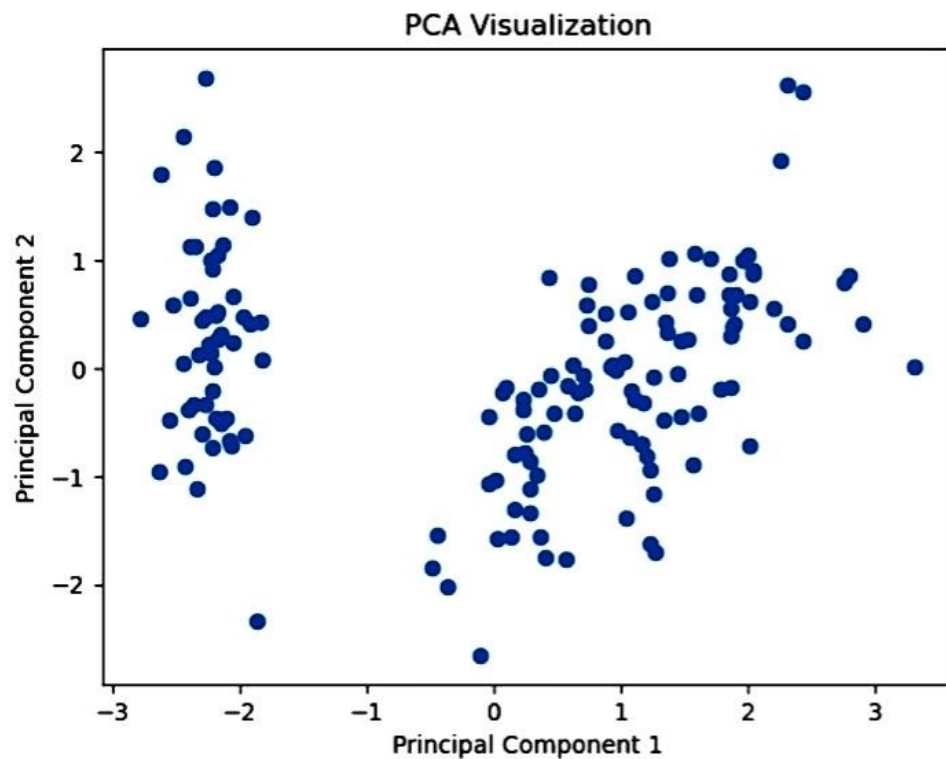
print(df.head())

# Scatter Plot
plt.scatter(X_pca[:,0], X_pca[:,1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualization")
plt.show()
```

Explained Variance Ratio:

```
[0.72962445 0.22850762]
```

	PC1	PC2
0	-2.264703	0.480027
1	-2.080961	-0.674134
2	-2.364229	-0.341908
3	-2.299384	-0.597395
4	-2.389842	0.646835

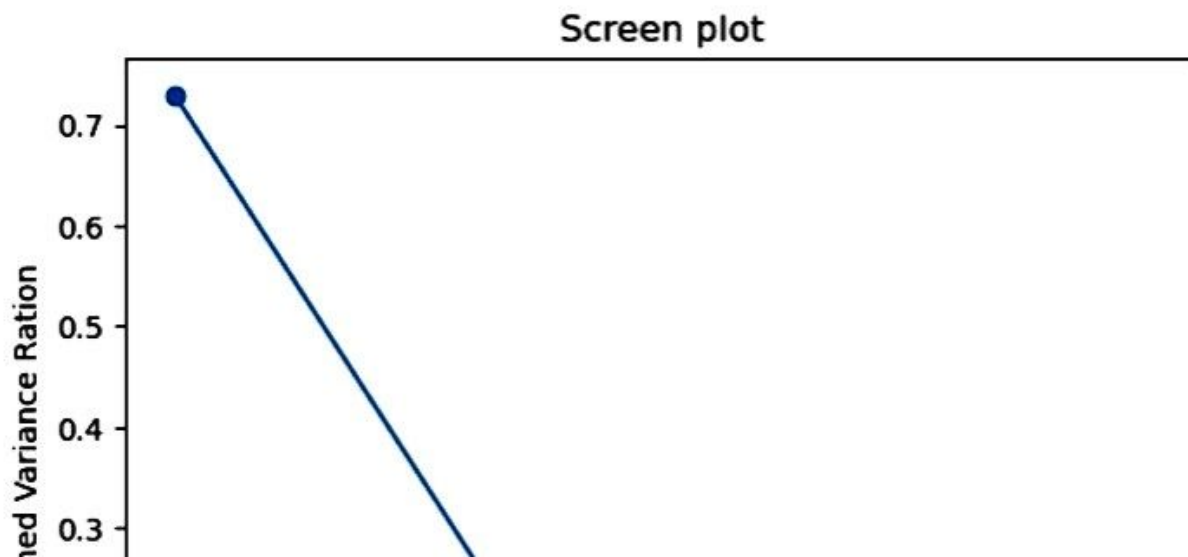


```
[2]: #Screen plot
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

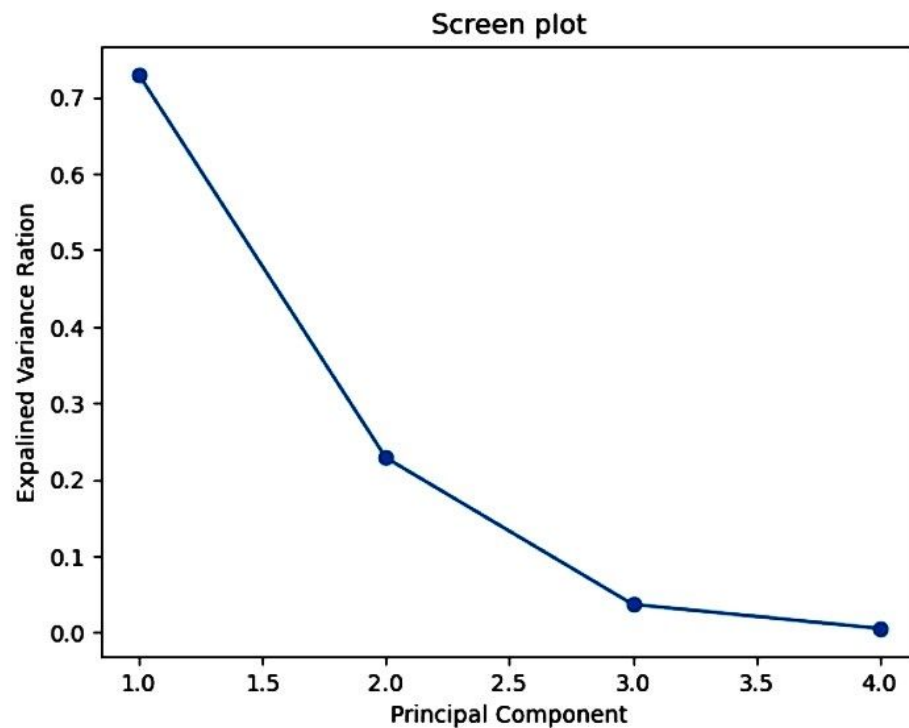
pca=PCA()
pca.fit(X_scaled)

plt.plot(
    range(1,len(pca.explained_variance_ratio_) +1),
    pca.explained_variance_ratio_,
    marker='o'
)

plt.xlabel("Principal Component")
plt.ylabel("Expalined Variance Ration")
plt.title("Screen plot")
plt.show()
```



```
plt.show()
```



▼ Cross Validation

- Cross validation is a technique used to evaluate a machine learning model by training and testing it on different subsets of the data.
- Its main purpose is to check how well the model generalizes to unseen data.

Why is a Single Train-Test Split Insufficient?

Suppose we split the data as:

- Training data = 80%
- Testing Data = 20%

problem

Performance depends on how the data was split.

- A lucky split may give very high accuracy.
- An unlucky split may give low accuracy.

Results can be misleading.

example

Split 1 -> Accuracy = 95% Split 2 -> Accuracy = % Split 3 -> Accuracy = %

K-Fold Validation ¶

In K-Fold Cross validation, the dataset is divided into K equal parts k=5 Fold1 Fold2 Fold3 Fold4 Fold5

Process

Iteration 1

Test -> Fold1 Train -> Fold2 + Fold3 + Fold4 + Fold5

Iteration 2

Test -> Fold2 Train -> remaining folds Continuous until every fold is used once as test data.

```
Fold1 = 92%  
Fold2 = 90%  
Fold3 = 94%  
Fold4 = 91%  
Fold5 = 93%
```

$(92+90+94+91+93)/5 = 92\%$

Benefits of K- fold Cross validation

- Uses all data for training and testing
- More reliable evaluation
- Reduces bias from a single train-test split
- Better estimate of model performance

Stratified k-Fold

Normal K-Fold may create folds with unequal class distribution.

Example Dataset

```
90 Pass  
10 Fail
```

Some folds may contain very few "Fail" samples

This can lead to unreliable evaluation

Stratified k-Fold solutions

Stratified k-Fold preserves

Example

Original Dataset 90% Pass 10% Fail

Each fold 90% Pass 10% Fail The same class ratio is maintained in every fold.

Benefits of Stratified K-Fold

- More reliable evaluation
- Better for classification problems
- Handles imbalanced datasets
- Maintains class proportions

K-Fold vs Stratified K-Fold

K-Fold	Stratified K-Fold
Random split into K folds	Preserves class distribution
Suitable for balanced data	Suitable for imbalanced data
May create uneven classes	Keeps class proportions same
General-purpose	Mostly used in classification

[3]: *#K-Fold Cluster*

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import KFold, cross_val_score
```

```
# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Model
model = DecisionTreeClassifier()

# K-Fold
kf = KFold(

    n_splits=5,

    shuffle=True,

    random_state=42

)

scores = cross_val_score(

    model,

    X,

    y,

    cv=kf

)

print("Scores:", scores)
print("Mean Accuracy:", scores.mean())
```

Scores: [1. 0.96666667 0.93333333 0.93333333 0.93333333]
Mean Accuracy: 0.9533333333333335

```
[4]: #stratiedKFold

from sklearn.datasets import load_iris

from sklearn.tree import DecisionTreeClassifier

from sklearn.model_selection import StratifiedKFold, cross_val_score

# Load dataset

iris = load_iris()

X = iris.data

y = iris.target

# Model

model = DecisionTreeClassifier()

# Stratified K-Fold

skf = StratifiedKFold(

    n_splits=5,

    shuffle=True,

    random_state=42

)
```

```

shuffle=True,

random_state=42

)

scores = cross_val_score(

    model,

    X,

    y,

    cv=skf

)

print("Scores:", scores)

print("Mean Accuracy:", scores.mean())

```

```

Scores: [1.          0.96666667 0.93333333 0.96666667 0.9        ]
Mean Accuracy: 0.9533333333333335

```

Hyperparameter Tuning

Hyperparameter Tuning is the process of finding the best hyperparameter values for a machine learning model to improve performance.

What are Hyperparameters

Hyperparameters are settings that are chosen before training the model.

Examples

Hyperparameters are settings that are chosen before training the model.

Examples

Algorithm	Hyperparameters
KNN	n_neighbors (K)
Decision Tree	max_depth, min_samples_split
Random Forest	n_estimators, max_depth
SVM	C, kernel, gamma

Example

For KNN: $K = 3$ $K = 5$ $K = 7$

The value of K is a hyperparameter.

Why hyperparameter Tuning

Different hyperparameter values can produce different results.

Example

$K = 1 \rightarrow \text{Accuracy} = 85\%$

$K = 3 \rightarrow \text{Accuracy} = 90\%$

$K = 5 \rightarrow \text{Accuracy} = 95\%$

The model performance changes when the hyperparameter changes.

Goal

Find the best combination of hyperparameters that gives the highest model

Goal

Find the best combination of hyperparameters that gives the highest model

Benefits

- Improves model accuracy
- Reduces overfitting
- Improves generalization
- Helps find the optimal model configuration

▼ Hyperparameter tuning methods

1. GridSearchCV

GridSearchCV tries all possible combinations of hyperparameter and selects the combination with the best performance.

Example

Suppose: `max_depth = [3,5,7]` `min_sample_split = [2,4]`

GridSearchCV tests:

All combinations are evaluated using Cross Validation.

Advantages ¶

- Finds the best combination from the given grid
- Easy to understand
- Easy to implement

2. RandomizedSearchCV

RandomizedSearchCV randomly selects a fixed number of parameter combinations instead of testing all combinations.

RandomizedSearchCV may test: 10 random combinations instead of all 100

Advantages

- Faster than RandomizedSearchCV
- Works well with large parameter spaces
- Lower computational cost

Disadvantages

- May miss the absolute best combination

3. Optuna Basics

Optuna is a modern hyperparameter optimization library.

- Instead of checking all combinations or random combinations ,it learns from previous trials and intelligently chooses better hyperparameter.

How Optuna works

```
Trial 1
|
Evaluate Model
|
Learn results
|
Suggest Better Parameters
|
Trial 2
```

This process continues until the best parameters are found.

Advantages

- Faster optimization
- Smaller search strategy
- Can stop poor-performing trials early (Pruning)
- Widely used in machine learning and deep learning

Disadvantages

- Requires installing an additional library
- Slightly more complex than GridSearchCV

Comparison

Method	Search Type	Speed
GridSearchCV	Tests all combinations	Slow
RandomizedSearchCV	Tests random combinations	Faster
Optuna	Learns and chooses better combinations	Fastest

```
[5]: pip install optuna
```

Collecting optunaNote: you may need to restart the kernel to use updated packages.

Downloading optuna-4.9.0-py3-none-any.whl.metadata (15 kB)

Requirement already satisfied: alembic>=1.5.0 in c:\users\harsh\anaconda3\lib\site-packages (from optuna) (1.13.3)

Collecting colorlog (from optuna)

Downloading colorlog-6.10.1-py3-none-any.whl.metadata (11 kB)

Successfully installed colorlog-6.10.1 optuna-4.9.0

```
[11]: ## GridSearchCV code
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# dataset
iris=load_iris()
X= iris.data
y= iris.target

#model
model = DecisionTreeClassifier()

# Parameter grid
param_grid = {
    'max_depth': [2,3,4,5],
    'min_samples_split': [2,4,6]}

# Grid search
grid = GridSearchCV(
    model,
    param_grid,
    cv=5,
    scoring='accuracy'
)
grid.fit(X,y)
print("Best Parameters:",grid.best_params_)
print("Best Score:",grid.best_score_)

Best Parameters: {'max_depth': 3, 'min_samples_split': 6}
Best Score: 0.9733333333333334
```

Best Score: 0.9733333333333334

```
[17]: ## RandomizedSearchCV
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV

# dataset
iris=load_iris()
X= iris.data
y= iris.target

#model
model = DecisionTreeClassifier()

# Parameter grid
param_dist = {
    'max_depth': [2,3,4,5,6,7,8],
    'min_samples_split': [2,4,6,8,10]}

# Grid search
random_search = RandomizedSearchCV(
    model,
    param_distributions = param_dist,
    n_iter = 6,
    cv=5,
    random_state=42
)
random_search.fit(X,y)
print("Best Parameters:",random_search.best_params_)
print("Best Score:",random_search.best_score_)
```

Best Parameters: {'min_samples_split': 4, 'max_depth': 7}

Best Score: 0.9666666666666668

[18]: !pip install optuna

```
Requirement already satisfied: optuna in c:\users\harsh\anaconda3\lib\site-packages (4.9.0)
Requirement already satisfied: alembic>=1.5.0 in c:\users\harsh\anaconda3\lib\site-packages (from optuna)
Requirement already satisfied: colorlog in c:\users\harsh\anaconda3\lib\site-packages (from optuna)
Requirement already satisfied: numpy in c:\users\harsh\anaconda3\lib\site-packages (from optuna)
Requirement already satisfied: packaging>=20.0 in c:\users\harsh\anaconda3\lib\site-packages (from optuna)
Requirement already satisfied: sqlalchemy>=1.4.2 in c:\users\harsh\anaconda3\lib\site-packages (from optuna)
Requirement already satisfied: tqdm in c:\users\harsh\anaconda3\lib\site-packages (from optuna) (4)
Requirement already satisfied: PyYAML in c:\users\harsh\anaconda3\lib\site-packages (from optuna)
Requirement already satisfied: Mako in c:\users\harsh\anaconda3\lib\site-packages (from alembic>=1)
Requirement already satisfied: typing-extensions>=4 in c:\users\harsh\anaconda3\lib\site-packages
Requirement already satisfied: greenlet!=0.4.17 in c:\users\harsh\anaconda3\lib\site-packages (from sqlalchemy>=1.4.2)
Requirement already satisfied: colorama in c:\users\harsh\anaconda3\lib\site-packages (from colorlog)
Requirement already satisfied: MarkupSafe>=0.9.2 in c:\users\harsh\anaconda3\lib\site-packages (from Mako)
```

[22]: **import** optuna
from sklearn.datasets **import** load_iris
from sklearn.tree **import** DecisionTreeClassifier
from sklearn.model_selection **import** cross_val_score

```
#datasets
iris=load_iris()
X=iris.data
y=iris.target
```

```
#objective
def objective(trial):
    max_depth=trial.suggest_int(
        "max_depth",
        2,
        10
    )
```

```
#objective
def objective(trial):
    max_depth=trial.suggest_int(
        "max_depth",
        2,
        10
    )

    min_samples_split=trial.suggest_int(
        "min_samples_split",
        2,
        10
    )

    model=DecisionTreeClassifier(
        max_depth=max_depth,
        min_samples_split=min_samples_split
    )

    score=cross_val_score(
        model,
        X,
        y,
        cv=5
    ).mean()
    return score

#Study
study=optuna.create_study(
    direction="maximize"
)

study.optimize(
    objective,
```




```
)

study.optimize(
    objective,
    n_trials=20
)

print("Best parameters:")
print(study.best_params)

print("Best score:")
print(study.best_value)
```

[I 2026-06-12 12:35:23,973] A new study created in memory with name: no-name-bbe493a0-902f-47f7-99f1-87ae34e0062e

[I 2026-06-12 12:35:23,993] Trial 0 finished with value: 0.9333333333333332 and parameters: {'max_depth': 2, 'min_samples_split': 10}. Best is trial 0 with value: 0.9333333333333332.

[I 2026-06-12 12:35:24,015] Trial 1 finished with value: 0.9666666666666668 and parameters: {'max_depth': 5, 'min_samples_split': 8}. Best is trial 1 with value: 0.9666666666666668.

[I 2026-06-12 12:35:24,034] Trial 2 finished with value: 0.96 and parameters: {'max_depth': 3, 'min_samples_split': 10}. Best is trial 1 with value: 0.9666666666666668.

[I 2026-06-12 12:35:24,048] Trial 3 finished with value: 0.9733333333333334 and parameters: {'max_depth': 3, 'min_samples_split': 5}. Best is trial 3 with value: 0.9733333333333334.

[I 2026-06-12 12:35:24,062] Trial 4 finished with value: 0.9600000000000002 and parameters: {'max_depth': 8, 'min_samples_split': 2}. Best is trial 3 with value: 0.9733333333333334.

[I 2026-06-12 12:35:24,078] Trial 5 finished with value: 0.9666666666666668 and parameters: {'max_depth': 6, 'min_samples_split': 6}. Best is trial 3 with value: 0.9733333333333334.

[I 2026-06-12 12:35:24,093] Trial 6 finished with value: 0.9600000000000002 and parameters: {'max_depth': 10, 'min_samples_split': 4}. Best is trial 3 with value: 0.9733333333333334.

[I 2026-06-12 12:35:24,106] Trial 7 finished with value: 0.9666666666666668 and parameters: {'max_depth': 4, 'min_samples_split': 8}. Best is trial 3 with value: 0.9733333333333334.

[I 2026-06-12 12:35:24,119] Trial 8 finished with value: 0.9600000000000002 and parameters: {'max_depth': 8, 'min_samples_split': 3}. Best is trial 3 with value: 0.9733333333333334.

[I 2026-06-12 12:35:24,131] Trial 9 finished with value: 0.9533333333333334 and parameters: {'max_depth': 8, 'min_samples_split': 2}. Best is trial 3 with value: 0.9733333333333334.

[I 2026-06-12 12:35:24,144] Trial 10 finished with value: 0.9666666666666668 and parameters: {'max_depth': 10, 'min_samples_split': 5}. Best is trial 3 with value: 0.9733333333333334.

```
[I 2026-06-12 12:35:24,131] Trial 9 finished with value: 0.9533333333333334 and parameters: {'max_depth': 8, 'min_samples_split': 2}. Best is trial 3 with value: 0.9733333333333334.
[I 2026-06-12 12:35:24,144] Trial 10 finished with value: 0.9666666666666668 and parameters: {'max_depth': 10, 'min_samples_split': 5}. Best is trial 3 with value: 0.9733333333333334.
[I 2026-06-12 12:35:24,159] Trial 11 finished with value: 0.9666666666666668 and parameters: {'max_depth': 5, 'min_samples_split': 7}. Best is trial 3 with value: 0.9733333333333334.
[I 2026-06-12 12:35:24,183] Trial 12 finished with value: 0.9666666666666668 and parameters: {'max_depth': 5, 'min_samples_split': 8}. Best is trial 3 with value: 0.9733333333333334.
[I 2026-06-12 12:35:24,204] Trial 13 finished with value: 0.9333333333333332 and parameters: {'max_depth': 2, 'min_samples_split': 6}. Best is trial 3 with value: 0.9733333333333334.
[I 2026-06-12 12:35:24,227] Trial 14 finished with value: 0.9666666666666668 and parameters: {'max_depth': 4, 'min_samples_split': 8}. Best is trial 3 with value: 0.9733333333333334.
[I 2026-06-12 12:35:24,257] Trial 15 finished with value: 0.96000000000000002 and parameters: {'max_depth': 6, 'min_samples_split': 4}. Best is trial 3 with value: 0.9733333333333334.
[I 2026-06-12 12:35:24,283] Trial 16 finished with value: 0.96 and parameters: {'max_depth': 3, 'min_samples_split': 9}. Best is trial 3 with value: 0.9733333333333334.
[I 2026-06-12 12:35:24,311] Trial 17 finished with value: 0.9666666666666668 and parameters: {'max_depth': 6, 'min_samples_split': 6}. Best is trial 3 with value: 0.9733333333333334.
[I 2026-06-12 12:35:24,340] Trial 18 finished with value: 0.9666666666666668 and parameters: {'max_depth': 4, 'min_samples_split': 5}. Best is trial 3 with value: 0.9733333333333334.
[I 2026-06-12 12:35:24,362] Trial 19 finished with value: 0.96 and parameters: {'max_depth': 3, 'min_samples_split': 7}. Best is trial 3 with value: 0.9733333333333334.
Best parameters:
{'max_depth': 3, 'min_samples_split': 5}
Best score:
0.9733333333333334
```